USAGE · 01 / 03 · 模板使用规则 + 内容要点

怎么用PPT模板 + 课件内容硬要求

DACon 课件模板使用规范 & 内容审核底线要求

可修改

封面填写嘉宾姓名 + 演讲题目
目录页可增减小标题
正文页内容、图表、案例全部自由调整

→ 你的发挥主场

可保留

母版背景（最好不调整）
页脚 DACon · DataFun 双品牌 Logo
正文字号规范：标题 28–32pt / 正文≥18pt

→ 统一视觉节奏

请勿改动

封面主视觉背景
距离页面下方 50px 的嘉宾名字排版行
固定字体：黑体 / 微软雅黑 / Arial

→ 改了 = 看着不像同款

课件内容 · 6 条硬要求

时长 45 分
20-25 页（1.5-2 分/页），含 1-2 段 Q&A 时间。

结构 推荐链路
问题 → 方法 → 架构 → 案例 → 指标 → 踩坑。开篇先放目录。

案例 真实生产
DACon 观众偏好落地案例 > 理论；有真实数据 + 量化结论最佳。

少字多图
避免大段文字；优先架构图、流程图、数据可视化。

字号 保证后排看清
标题 28-32pt · 正文 18-24pt · 代码 ≥14pt。

整体 暗色调
新增元素避免高饱和亮色；模版适配会场屏幕 16:9 。

提交 speaker@datafun.pro · 邮件标题固定格式：DACon2026-姓名-版本 · 9.30 初稿 / 10.18 终稿

课件制作 5 阶段流程 | 全流程关键时间节点总览

整体流程从初稿打磨至大会现场，核心锁稿节点： **9.30 初稿** 和 **10.18 终稿**，把 V1 / V2 列入日历；试讲 + 脱敏跟着走。

TODAY 当前进度 8.10 • W3 • 已投过/正在打磨议题

5 步流程 • 从截稿到脱敏

STEP ①
初稿提交
9 月 30 日

截止当天 23:59
整体结构完整，缺漏可标"待终稿补充"

STEP ②
审核反馈
9.30 → 10 月

专家委员会 1v1 反馈
含结构 / 案例 / 可读性

STEP ③
终稿定稿
10.18

已按反馈全量修改
字号 / 间距 / 比例复核

STEP ④
线上试讲
10.14–10.17

用最终版课件
不接受临时替换

STEP ⑤
脱敏版提交
10.21 前

案例全部脱敏
用于**公开回放 + DataFun 长文**

关键节点 • 一行看完 6 个固定日期

8.11 现阶段	9.30	10.14–17	10.18	10.21	10.23–24
W3 · 已投过/打磨议题	初稿提交 截止当日	线上试讲 专家 1v1	终稿定稿 通过审核	脱敏版提交 公开稿件	现场 40 分钟 大现场演讲

✉ 提交 **DACon2026-张三-v1** 发到 **speaker@datafun.pro**；同一份 PPT 文件后缀要对应版本号。

EVENT · 03 / 03 · 大会现场周流程 & 讲师现场硬性规则

10.22 统一 PPT 设备测试日 · 当周 10.19–10.24

10.22 由组委会统一搭建、调试主论坛 + 全部 15 个分论坛现场设备 — **讲师无需到场**。你真正要做的是：**测试前把终稿、没有再修改的课件给到组委会**（10.18 终稿之后**不接受临时替换**）。

时间	地点	规模	架构	讲师时长
10.23 –10.24	北京希尔顿逸林酒店	800-1200 人	1个主论坛+ 15 个分论坛	45 分钟
周五 – 周六 · 2 天	各分论坛详细会场地址会前单独同步	精英汇聚	您分配至 Layer1~Layer4 分论坛	含 Q&A

W13 · 当周重点
10.19–10.21 本地备课 · 课件预热
10.22 全场 PPT 测试 · 主论坛 + 15 分论坛
10.23 上午 主论坛 / 演讲
10.23/24 下午 分论坛 / 演讲 · 专场互动 · 茶歇
10.24 收尾 反馈 + 复盘 · 物料归档

演讲时长 + 现场规则

45

演讲时长 · SLOT

40 分钟正式内容分享
5–10 分钟现场 Q&A 问答
全程严格控制在 50 分钟以内，超时将现场提醒截断

现场规则 · 4 条

演讲 40 分钟 + Q&A 5–10 分钟，全程 50 分钟以内
到场签到要求：您的场次开场前 15 分钟抵达对应分会场，完成话筒、翻页器调试
终稿 PPT 会前统一交付组委会预存会场主机；禁止使用外接 U 盘、网页外链、未嵌入本地视频，避免播放失败、页面乱码。
本周 白天 / 备课 / 锁版

当周重点 10.19–10.21 备课 / 预热 · 10.22 全部论坛 PPT 测试 · 10.23–10.24 现场演讲 · 10.24 反馈+复盘

提醒 10.18 终稿之后不接受临时替换 · 测试日 10.22 全部 15 个分论坛由组委会统一开机调试，讲师无需到场。

以下为演讲使用的PPT模版

从通用到专用
小红书面向搜索生成式召回的推理选型与优化

演讲嘉宾：骆兆楷

目录

CONTENT

01 背景介绍

02 业务痛点

03 方案对比与选型逻辑

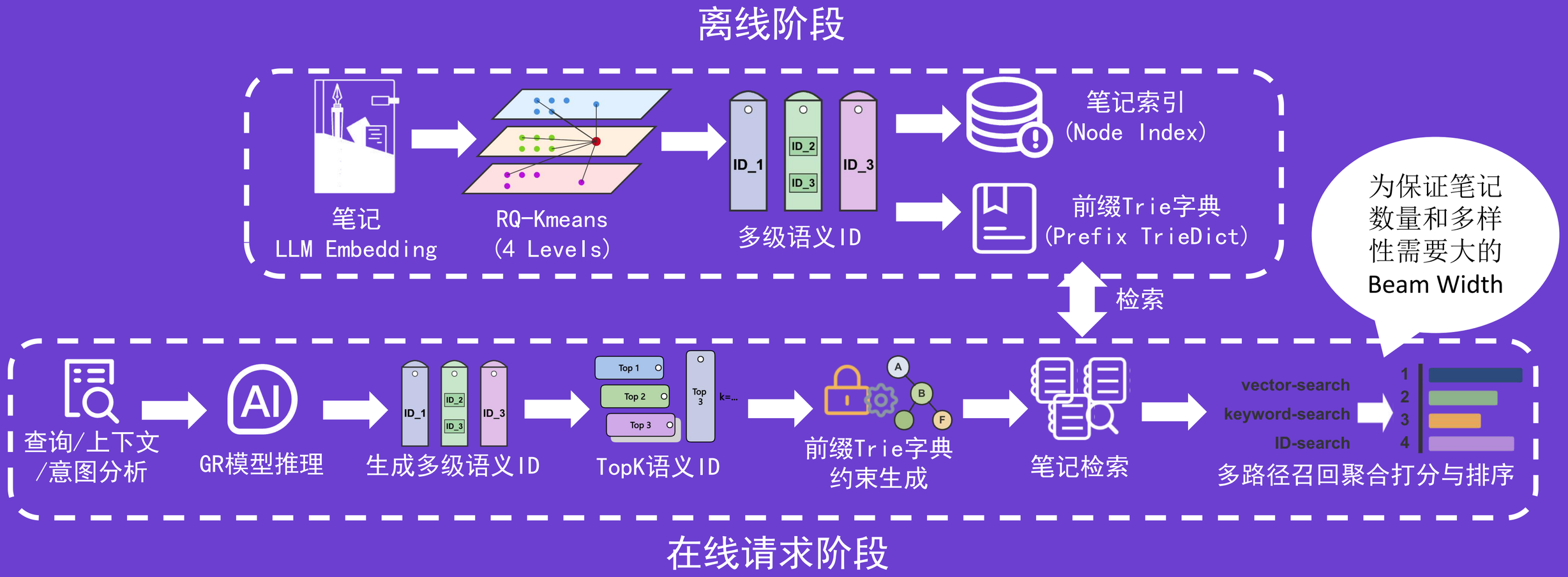
04 GR-Inference架构与核心实现

05 实践落地量化效果

06 经验总结与未来方向

01 背景介绍

背景介绍 模型-业务链路


离线阶段
笔记
LLM Embedding
RQ-Kmeans
(4 Levels)
ID_1
ID_2
ID_3
多级语义ID
笔记索引
(Node Index)
前缀Trie字典
(Prefix TrieDict)
检索
为保证笔记数量和多样性需要大的Beam Width
查询/上下文/意图分析
AI
GR模型推理
生成多级语义ID
Top 1
Top 2
Top 3
Top 3
k=...
TopK语义ID
A
B
F
前缀Trie字典约束生成
笔记检索
vector-search
keyword-search
ID-search
1
2
3
4
多路径召回聚合打分与排序
在线请求阶段

02 业务痛点

业务痛点　GR生成式推荐推理的数据和计算特点

Long Context

搜索 Query + 上下文

1K – 32K tokens

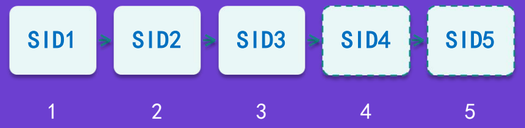


长序列输入

Short Decode

3 – 5 级 SID，每级一次 Decode

3 – 5 次 Decode



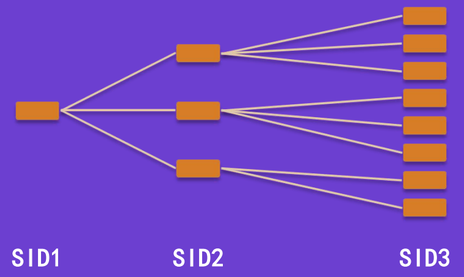
1 级 SID = 1 次 Decode

少步生成，逐级细化

Large Beam

Beam 宽度随 Decode 动态变化

20 → 300 → 900



多路径搜索，候选逐层扩展

业务痛点　通用推理框架在搜索场景的性能瓶颈

vLLM　SGL　TensorRT-LLM

01 独立Seq调度



KV / 带宽冗余

02 合批缺少Beam感知



Batch x Beam 低效

03 Beam外层循环调度



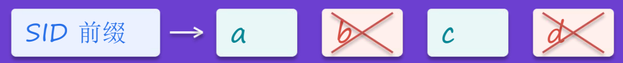
Req 组装开销

04 Beam-SID多级管理缺失



层间候选难管理

05 Item 约束过滤缺失



层内无效 SID 难过滤

1. 通用方案在搜索场景的GPU利用率不足40%
2. 单次搜索召回延迟影响用户体验

03 方案对比与选型逻辑

方案对比与选型逻辑

方案	Long Context支持	Short Decode优化	Large Beam支持	适用场景
vLLM	优秀	一般	不支持	通用对话
TensorRT-LLM	良好	较好	有限	通用推理
DeepSpeed-MII	良好	一般	不支持	大模型服务
GR-Inference（我们）	**优化**	**极致优化**	**原生支持**	**搜索召回**

方案对比与选型逻辑 我们的选型逻辑

vLLM · Beam Search

组装 Req → 推理 → 选 Beam

外层逐步循环 → 调度开销增加

TensorRT-LLM · Batching

3 - 5 步短 Decode → 动态大 Beam

通用合批仍需 GR 专用适配

搜索推理需求

高并发

低延迟

大 / 动态 Beam

选择自研 GR-Inference：围绕搜索负载深度优化

请求级共享 KV
降低大 Beam 访存冗余

Beam 感知合批
协同 Batch x Active Beam

专用 Decode 路径
CUDA Graph + GPU SID Trie

方案对比与选型逻辑 选型后的新问题



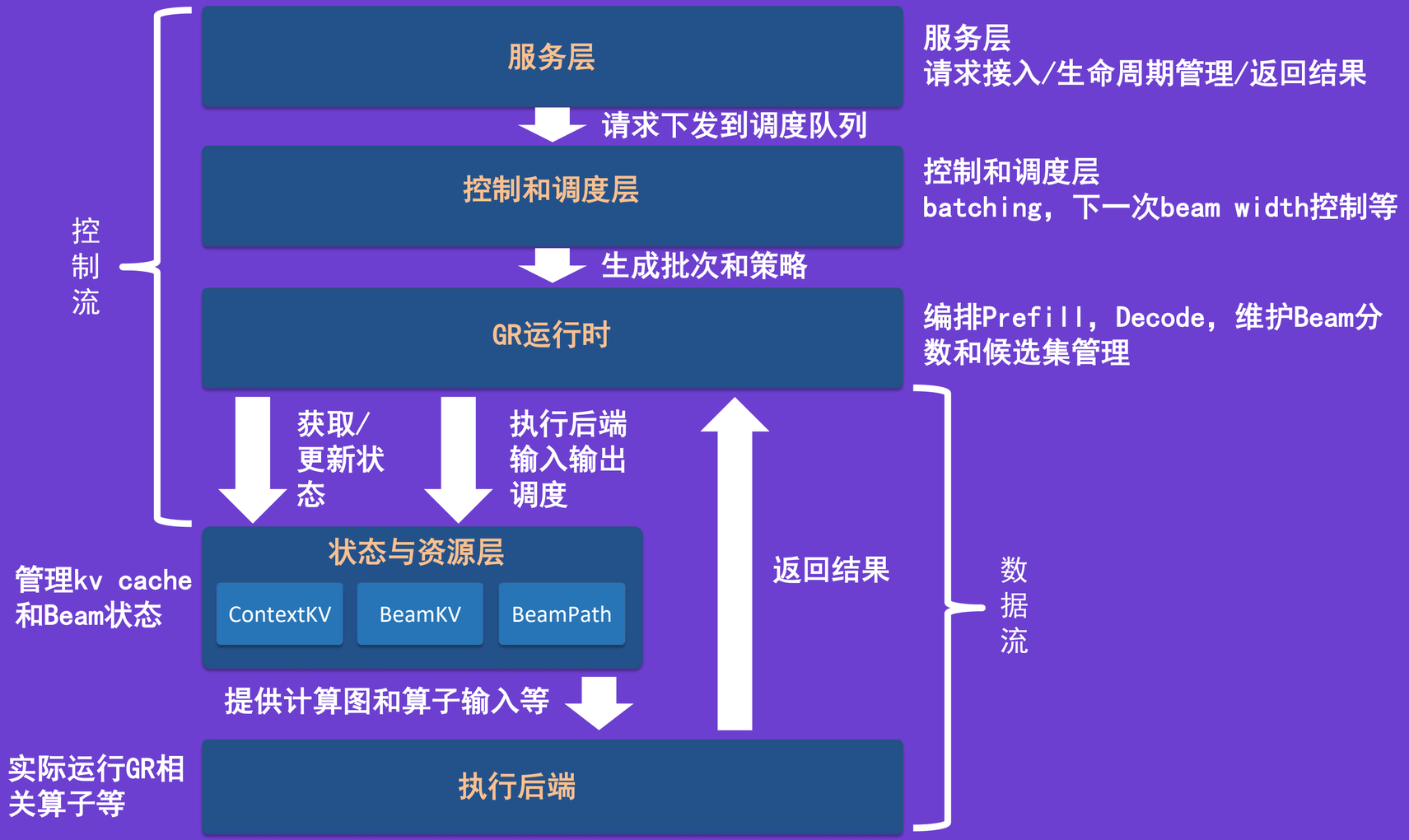
如何平衡通用性和专用性？

如何保证系统的可维护性和扩展性？

如何快速迭代适配业务变化？

04 GR-Inference架构与核心实现

GR-Inference架构与核心实现



GR-Inference架构与核心实现 核心优化1 - Long Context

KV类型	实际对应内容	特点
ContextKV	原始Query上下文	很长，多条Beam需要访问的ContextKV完全相同
BeamKV	前几级已生成的SID对应的KV	很短，各Beam的路径不同

GR-Inference架构与核心实现 核心优化1 - Long Context

假设搜索请求有8K tokens的上下文、300条Beam，现在生成第三级SID



性能瓶颈：

1. ContextKV显存带宽浪费：显存访问量 = 300 * 8K tokens的KV
2. BeamKV访存&计算效率不高：短、分叉、不规则

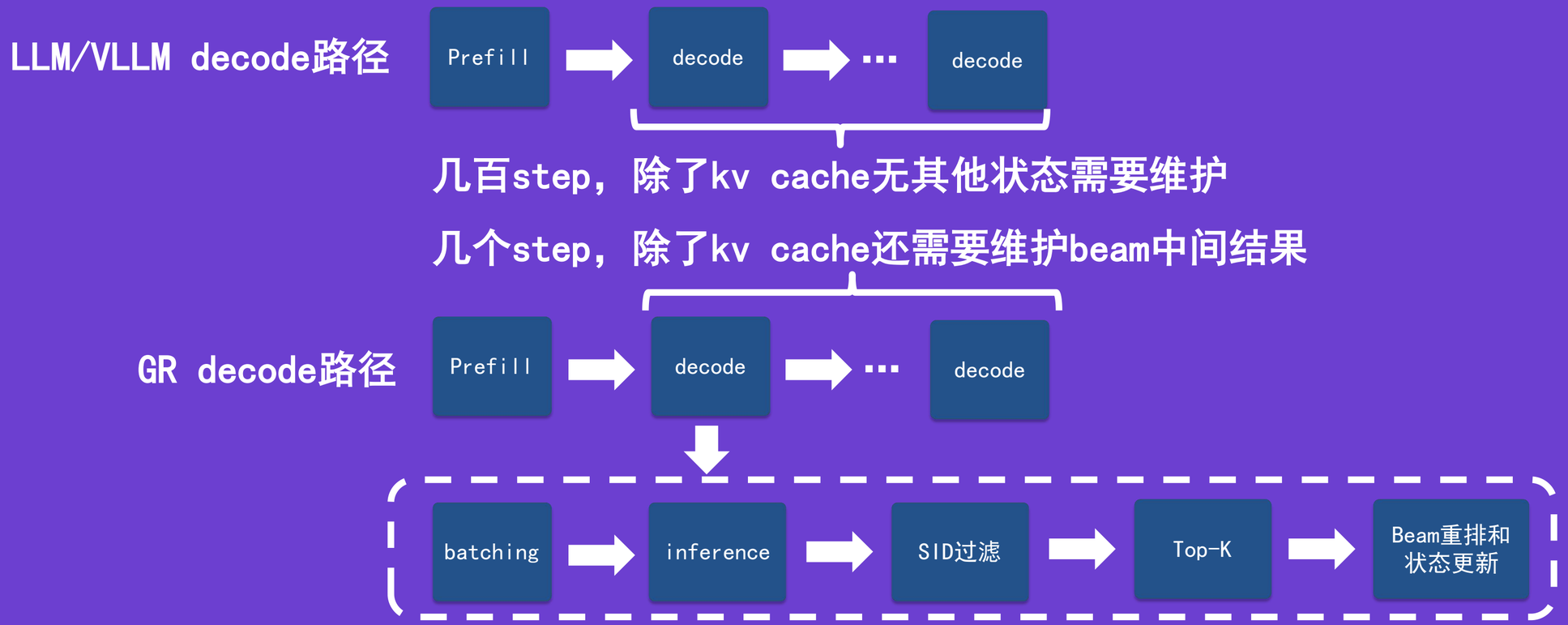
GR-Inference架构与核心实现 核心优化1 - Long Context

Kernel	资源层区块	功能	动机
K1	ContextKV	计算长Context的Attention	1. 用Tensor Core处理密集计算，复用共享上下文 2. 用Split-KV增加并行度
K2	BeamKV	前几级已生成的SID对应的KV	用轻量CUDA Core处理不规则Gather和短序列计算
K3	K1+K2输出	合并两段Attention结果	按统一的Softmax权重，得到最终输出

内存利用率提升 **60%**

支持并发数提升 **3倍**

GR-Inference架构与核心实现 核心优化2 - Short Decode

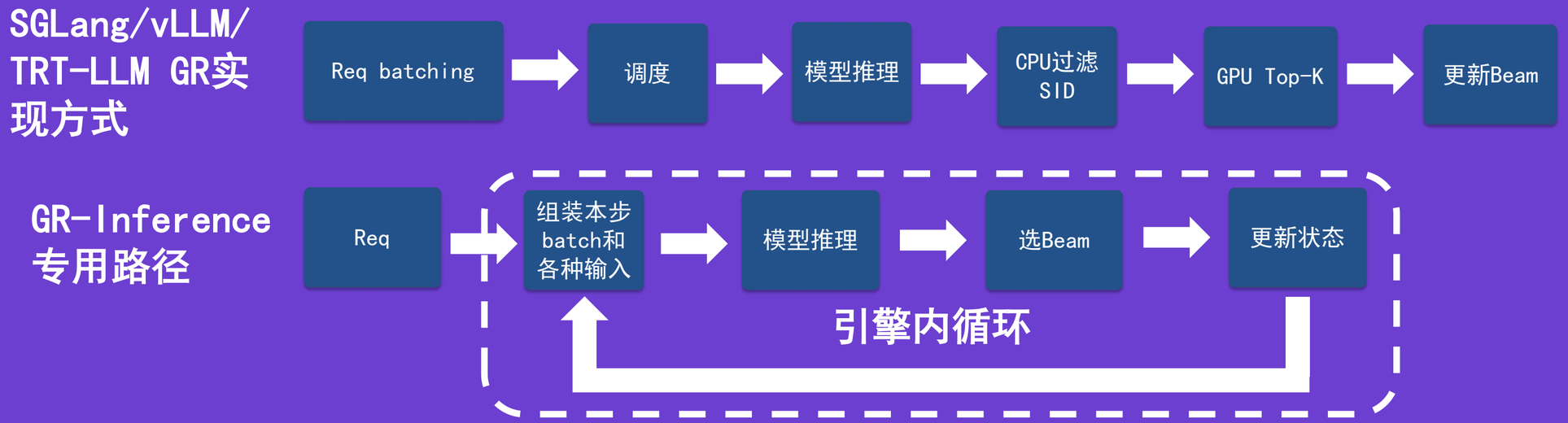


性能瓶颈：

1. Beam Search逐步依赖：必须先选出本层候选，才能知道下一层用哪些SID、父路径和Beam，SGLang外层循环实现容易反复组装请求、同步结果

2. Beam Width很大，且宽度会变化：一个请求可能维护数百条路径，每一步都要筛选、重排和更新状态，批次形状也随之变化

GR-Inference架构与核心实现 核心优化2 - Short Decode



“业务侧反复调用”变成“引擎内复用状态、逐步推进”，再把每一步的 GPU 执行开销压低

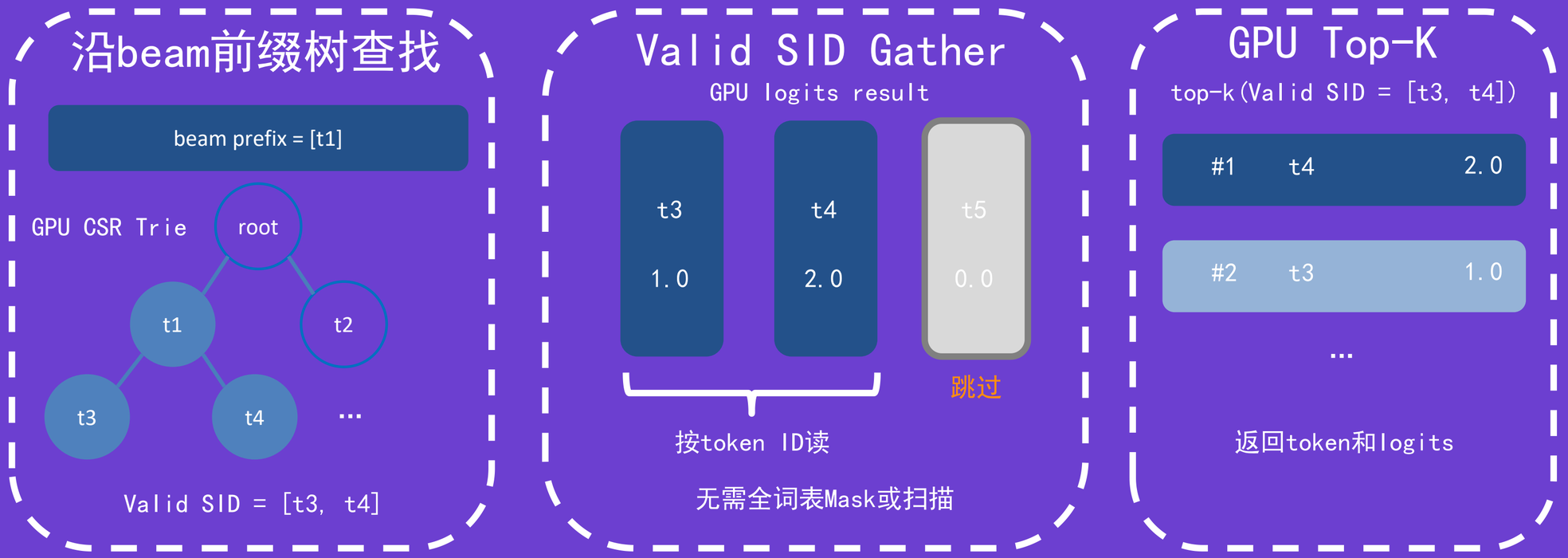
GR-Inference架构与核心实现 核心优化2 - Short Decode



做法	具体优化点
请求状态常驻	KV、Beam父子路径和候选状态留在引擎里，下一步直接接着用
执行DAG Pre-Capture	按batch×beam容量分桶捕获CUDA Graph，减少每一步逐个启动算子的开销
每步只更新元数据	更新长度、Pool Slot、Decode Step、活跃beam数等，让Pre-Capture DAG处理当前请求
候选筛选留在GPU GM	GPU Trie查询合法SID 后继并执行受限Top-K，减少CPU查树和CPU－GPU同步

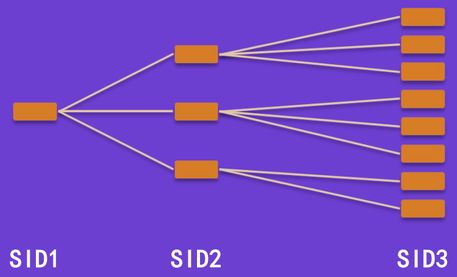
GR-Inference架构与核心实现 核心优化2 - Short Decode

GPU筛选合法SID和Top-K kernel fusion



GR-Inference架构与核心实现 核心优化3 - Large Beam支持

做法	具体优化点
用索引管理分叉	BeamKV保存短生成历史，BeamPath记录父子关系；扩展、重排和回溯通过索引完成，避免复制整段长Context
按当前Beam宽度调度	以请求管理状态，执行时按Batch × ActiveBeam展开；支持固定宽度、分层宽度，以及按分数差距收缩宽度
Batch × Beam分桶CUDA Graph	降低大批候选每一步执行时的算子启动开销



多路径搜索，候选逐层扩展

beam=1000时，延迟仅比beam=1增加20%（线性增长为10倍）

GR-Inference架构与核心实现 关键决策点



工程决策的核心是权衡——在性能、成本、可维护性之间找到最优解

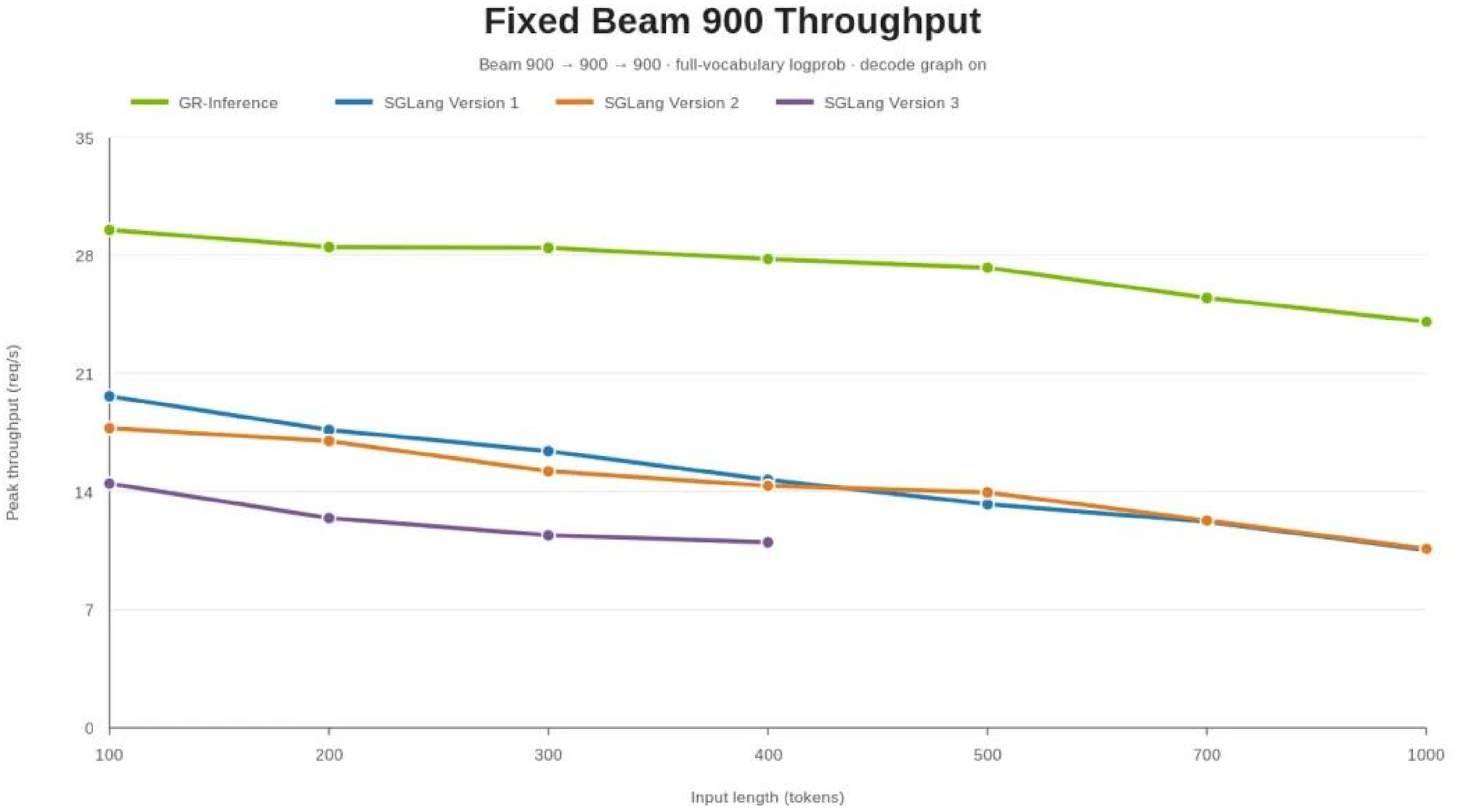
05 实践落地量化效果

实践落地量化效果 baseline SGL

版本	版本
Version 1	feature/beam_search, commit af4e3f42a5a1
Version 2	feature/beam_search_update_0801, commit 9380595f3392
Version 3	lsyin/beam-search-dev, commit 5203fab5eb97（Draft PR #31626）

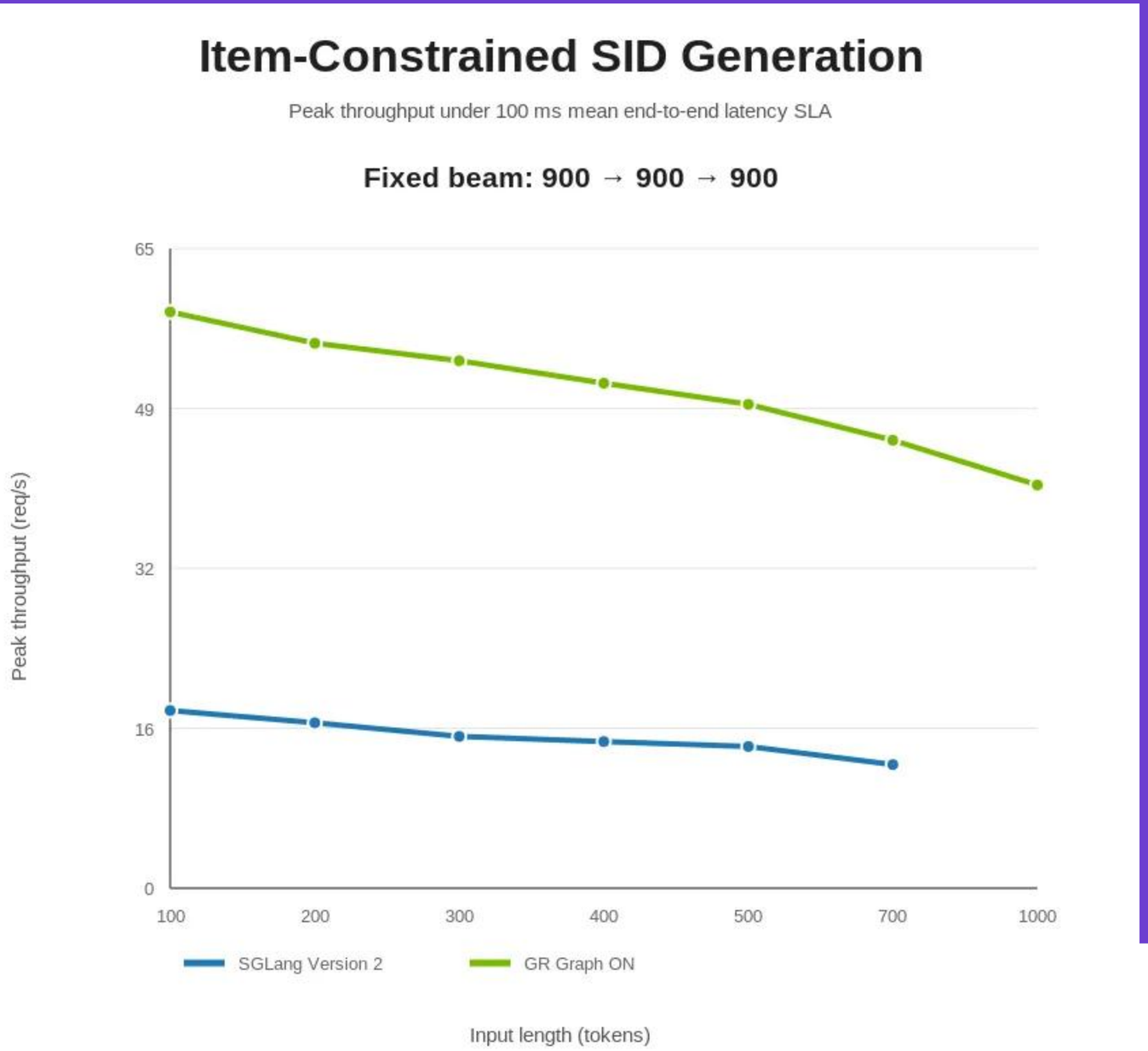
Qwen3-0.6B的模型，prefill长度从100逐步加到1000，decode长度固定为3，对比的是在E2E Latency不超过100ms的情况下各框架能达到的最大吞吐。

实践落地量化效果 baseline SGL



第一组测试将三步 Beam 都固定为 900。GR-Inference 和三套 SGLang 均启用 Decode CUDA Graph，并统一使用普通的全词表 logprob。

实践落地量化效果 baseline SGL



第二组测试同样将三步 Beam 都固定为 900，比较 GR-Inference 与唯一支持该功能的 SGLang Version 2，两边均启用 Decode Graph。

GR-Inference 吞吐是 SGLang Version 2 的 3.24～3.63 倍

实践落地量化效果

在固定Beam 900的SLA口径下，GR-Inference相对三套SGLang社区实现取得1.5x - 2.5x收益，Item-constrained场景相对SGLang提升3.2x以上。

GR-Inference在小红书搜索场域完成落地，召回阶段新增一路自回归SID召回通道，在耗时达标的情况下，实现了GPU资源的高效利用，最终项目顺利上线，点击率提升0.03pt，有效点击率提升0.2%，离线Recall@1000召回率提升5.7%。

06 经验总结与未来方向

经验总结与未来方向

1. KV Cache内存优化：面向真实Context长度分布引入多 Context Bucket 与 Paged ContextKV，让 Decode Attention 原生支持 Page Table，降低变长上下文的显存浪费。

2. CUDA Graph覆盖扩大：扩充Prefill/Decode Graph的预热形状，并将Beam Selection（log_softmax + Top-K）纳入 Graph，进一步缩短热路径。

3. 框架和算子优化：支持PD分离、多种并行策略扩展以支持模型scaling，做更细粒度的megakernel融合。

THANKS

演讲嘉宾：骆兆楷